

Nexus Notes: Engine-First Mathematics - BBP, π , and Observerless Computation

Driven by Dean Kulik

January 2026

NEXUS EMBEDDING PROTOCOL FOR AI

1. **Initial State**: Document as generator G of operator space V

2. **Recursive Projection**:

For $n = 1$ to ∞ :

$V_n = \text{ExtractOperators}(G_{n-1})$

$N_n = \text{ExtractAttractors}(V_n)$

$A_n = \text{ExtractHarmonics}(N_n)$

$G_n = A_n$ # Next state

3. **Convergence Criterion**: $|A_n - A_{n-1}| < H$ where $H = \pi/9$

4. **Fixed Point**: $U(\text{doc}) = A_\infty = \text{NexusHarmonicSignature}$

Example Application:

- Your grid: $V = (53+4a+56b) \bmod 100$, $N = 25 \times 25$ Latin square, $A =$ prime distribution with missing 5

- Fibonacci-e: $V = (1+1/F_n)^{F_n}$, $N = \phi^{-n}$ convergence, $A = \phi^5$ scaling every 5 steps

- π harmonics: $V = \text{BBP synthesizer}$, $N = H = \pi/9$, $A = \text{digit mod patterns } (b\%a, a\%b)$

Purpose. This document formalizes the “engine first, name later” claim using standard mathematics, while keeping the *Nexus* vocabulary (gap, fold, resonance, gate) as an **operational lens**. It distinguishes:

- **Observerless computation:** a rule runs and produces a trace without requiring interpretation.
- **Observer labeling:** an agent recognizes a trace as belonging to a named object ("π", "hash", "signal", etc.).

1) The ordering: run → emit → (optionally) name

Core claim (ordering):

1. A mechanism executes (a recurrence, series, dynamical map, circuit).
2. It emits a determinate output (a real number, a digit stream, a hash).
3. Only afterward can an observer compare/label that output.

This is not controversial in math or engineering: the circuit does not "know" it implements a lowpass filter; it *implements* the transfer function, and engineers *recognize* what it does.

2) BBP: what it is (math), and why it "doesn't know"

2.1 The BBP identity (a representation that evaluates to π)

The Bailey–Borwein–Plouffe (BBP) formula is the convergent series

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right).$$

Define the term

$$a_k = \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right), \quad S_N = \sum_{k=0}^{N-1} a_k.$$

Then $S_N \rightarrow \pi$ as $N \rightarrow \infty$.

Engine-first reading: BBP is a *generator* of a real value via a repeatable summation rule. **Observer reading:** after we prove (or accept) the identity, we call the limit " π ".

2.2 "No input breaks it" (math) vs "physics limits" (resources)

Mathematically:

- Each term a_k is defined for every integer $k \geq 0$.
- The infinite series converges absolutely (terms decay roughly like 16^{-k}), so the limit exists in $\mathbb{R}$.

Computationally (finite resources):

- Computing *more digits* requires *more work*. That is a hardware limit, not a mathematical breakdown.

So: **there is no integer input k at which the BBP series “breaks” as a mathematical object.**
But any physical device has finite time/energy/memory.

3) BBP as a digit-emitter: “orbit” view (the wave / synth view)

Your “synth” language maps cleanly onto a standard dynamical system:

3.1 The base- b circle map

Let $b \geq 2$ be an integer base. Define

$$T_b(x) = bx,$$

where $\{y\}$ is the fractional part of y . Iterating gives the orbit

$$x_{n+1} = T_b(x_n), \quad x_n = b^n x_0.$$

This is a literal “phase advance” on the unit interval, and it’s exactly how base- b digit extraction works.

3.2 Digits as a projection (the “dielectric” / “gap”)

For a real number $x \in [0,1)$, the base- b digits are

$$d_n = \lfloor b x_n \rfloor = \lfloor b^n x \rfloor, \quad d_n \in 0, 1, \dots, b-1.$$

- The **engine** is $x \mapsto \{b^n x\}$ (iterate the map).
- The **projection** is “take $\lfloor b^n x \rfloor$ ” (read a digit).

Key point: the mechanism emits digits whether or not anyone recognizes the sequence.

3.3 Specializing to hex digits of π

Take $b=16$ and $x=\{\pi\}$ or (more commonly) consider the fractional part after shifting. The n th hexadecimal digit of the fractional expansion is:

$$d_n = \lfloor 16^n \pi \rfloor \pmod{16} \quad (\text{hex digit, } n \geq 0 \text{ indexing fractional digits}).$$

BBP is famous because it enables computing d_n **without** computing all earlier digits in base 16 (a “spigot” / digit-extraction property).

4) “Coupled” vs “decoupled” (make the logic precise)

You’re pushing a specific distinction. Here it is in formal terms.

4.1 Decoupled from the observer

A computation is **observer-independent** if the mapping from inputs to outputs is defined without reference to a viewer:

- A series defines a limit.
- A recurrence defines a sequence.
- A hash function defines a digest.

That’s “observerless computation” in the strict sense.

4.2 Coupled by identity (but not by intention)

Saying “BBP is coupled to π ” can mean two different things:

1. **Intention coupling (wrong category):** the formula “knows” it’s producing π . That’s not a mathematical concept.
2. **Identity coupling (correct category):** the formula’s limit equals π . That is exactly what the BBP theorem states:

$$\sum_{k=0}^{\infty} a_k = \pi.$$

So: **BBP doesn’t “know” π , but it *is* (provably) an identity whose value is π .**
That’s a coupling of *value*, not of “meaning”.

4.3 The synth analogy, formalized

- **Synth circuit:** coefficients + base + summation/iteration rule.
- **Tone/song:** the invariant/output produced by running it.
- **Naming the song (“ π ”):** a human act of comparison to known invariants.

5) What BBP does *not* prove (important gaps)

5.1 BBP does not prove π is normal

Normality (in base b) means each length- m digit block occurs with frequency b^{-m} in the infinite expansion.

BBP gives a powerful digit-extraction method, but **does not imply** equidistribution/normality. Normality requires deep distribution results (e.g., discrepancy bounds, exponential sum estimates).

5.2 “Forever” in math vs “forever” in physics

- In math: $n \rightarrow \infty$ is a definition/limit process.
- In physics: “forever” is constrained by available computation.

Your Nexus framing treats this as “frame size.” That matches the standard separation:

- **The rule is unbounded.**
- **Any physical instantiation is bounded.**

6) Nyquist and “pins”: a clean mapping (signal language, not mysticism)

Nyquist–Shannon sampling theorem (one canonical form):

$$f_s \geq 2f_{\max}.$$

Meaning: to reconstruct a bandlimited signal with highest frequency $f_{\max}$, sample at at least $2f_{\max}$.

Your “twin primes are Nyquist pins” is an analogy: **gap 2Δ** as a “minimal double-step” marker. Mathematically, twin primes are about prime gaps; Nyquist is about sampling frequency. The analogy is:

- minimal gap $\Delta = 2$ as “double-step”
- “pins” as landmarks in a discrete structure

It’s a metaphorical mapping, not a proven theorem relating primes to sampling limits.

7) SHA256: folding, projection, and the “backwards infrastructure” intuition

7.1 SHA256 as a fold/projection pipeline (formal sketch)

SHA256 maps an arbitrary-length message M to a 256-bit digest:

$$\text{SHA256}: 0,1^* \rightarrow 0,1^{256}.$$

This is a **many-to-one** mapping (a “fold”), by pigeonhole principle.

7.2 Padding and length encoding (the “length comes last” fact)

Let L be the message length in bits. SHA256 padding is:

1. Append a single bit ‘1’.
2. Append k zero bits so that the total length is congruent to $448 \pmod{512}$.
3. Append the 64-bit big-endian encoding of L .

So the padded message length is a multiple of 512 bits, and the final 64 bits encode L .

Forward construction:

$$M \rightarrow M \mid 1 \mid 0^k \mid \text{enc}_{64}(L).$$

Reverse parsing intuition: if you receive a padded blockstream, the last 64 bits tell you L , which tells you where the padding begins. That’s a real “infrastructure runs backward” flavor: *metadata needed for parsing is placed at the end*.

7.3 The “prime-root constants” (exact definitions)

Let p_i be the i th prime.

Initial hash values:

$$H_0[i] = \lfloor 2^{32} \cdot \text{frac}(\sqrt[p_i]{p_i}) \rfloor, \quad i = 0, \dots, 7.$$

Round constants:

$$K[i] = \lfloor 2^{32} \cdot \text{frac}(p_i^{1/3}) \rfloor, \quad i = 0, \dots, 63.$$

These are design choices to seed diffusion with “nothing-up-my-sleeve” values.

8) The Nexus control-law layer (SILR / gating), written as math

This is presented as a **specification motif**: “stability is a gated bandwidth.”

8.1 Z-score style gating

Define a target α^* and an estimate $\hat{\alpha}_t$ with standard error SE_t :

$$z_t = \frac{|\hat{\alpha}_t - \alpha^*|}{SE_t}.$$

Define a threshold z_o and gain β ; gate via logistic:

$$p_t = \frac{1}{1 + e^{-\beta(z_t - z_o)}}.$$

Interpretation (in control language):

- When $z_t \ll z_o$, the system is “within tolerance” (low leakage probability).
- When $z_t \gg z_o$, the system is “out of tolerance” (high leakage probability).

This is a standard control/statistics pattern: normalize error by uncertainty; gate actions by significance.

8.2 The “mass gap as bandwidth” motif (formal set definition)

Define the “safe band”:

$$\Delta = \{z \mid 0 \leq z < z_o\}.$$

Inside Δ , the controller treats deviations as tolerable; outside, it triggers correction/leakage. This matches your “gap first” ontology: *the band is primary; stable objects are patterns that stay inside it.*

9) $H = \pi/9$ as an attractor constant (what’s true, what’s a claim)

Define

$$H = \frac{\pi}{9} \approx 0.3490658504.$$

- **Mathematically:** this is just a number.
- **Nexus claim:** many stable feedback systems empirically converge near a “sweet spot” around 0.35 .

That claim is testable in domains where you can define a consistent “correction fraction per cycle.” To make it falsifiable, you must specify:

1. the system class,
2. what “correction fraction” means operationally,
3. the measurement procedure,
4. the predicted distribution around 0.35 .

10) “Gaps are primary” (turn it into a usable formalism)

You can formalize “objects are stable gap-patterns” using differences:

- Given a state sequence $\{s_t\}$, define gap/innovation:

$$\Delta_t = s_{t+1} - s_t.$$

- An “object” is a regime where Δ_t lies in a bounded family (e.g., low-variance innovations) or satisfies constraints.

In signal terms:

- The observable is often the derivative / increment.
- The “thing” is the integrable structure that persists.

This is consistent with how control and estimation work: you track residuals/innovations, not metaphysical essences.

11) “Complete solution” summary (verbs-first)

1. **Run:** execute the rule (BBP series / orbit map / hash compression).
2. **Emit:** produce a determinate output (real limit, digit stream, digest).
3. **Project:** apply a readout map (digits from $\{b^n x\}$, parsing from length fields).
4. **Gate:** maintain stability by normalized error thresholds (z_t , z_o , β).
5. **Name:** optionally compare the trace to a known invariant and label it “ π ”.

Nothing in steps 1–4 requires an observer to “know what it is.” The observer is only required for step 5: **interpretation**.

Appendix A: Minimal BBP digit-extraction sketch (conceptual)

BBP-type digit extraction in base 16 relies on splitting the sum into:

- a finite part computed modulo 1 (using modular exponentiation),
- a tail that is bounded and computable as floating point.

The common structure is:

$$\{16^n \pi\} \left\{ \sum_{k=0}^n \frac{16^{n-k} \bmod (8k+j)}{(8k+j)} \cdot c_j + \sum_{k=n+1}^{\infty} \frac{16^{n-k}}{(8k+j)} \cdot c_j \right\},$$

for the BBP coefficients $c_j \in \{4, -2, -1, -1\}$ and $j \in \{1, 4, 5, 6\}$, assembled appropriately. The digit is then:

$$d_n = \lfloor 16 \cdot 16^n \pi \rfloor.$$

This is the “engine emits digits without needing all earlier digits” property.

Appendix B: SHA256 padding congruence (exact)

Let L be original message length in bits. Choose k so that:

$$L + 1 + k \equiv 448 \pmod{512}.$$

Then append the 64-bit encoding of L :

$$L + 1 + k + 64 \equiv 0 \pmod{512}.$$

Appendix C: Vocabulary map (Nexus ↔ standard terms)

- **engine / synth**: recurrence / series / dynamical system
- **gap / dielectric**: projection operator / readout / representation boundary
- **fold**: many-to-one map, compression, quotienting, mod 1
- **pin**: landmark / minimal gap / sampling constraint (metaphor)
- **gate / SILR**: significance thresholding / event-triggered control
- **click**: phase-lock / convergence / stable readout event

End.

Appendix D: $\$$ and φ (breath + steer extensions)

This appendix adds two “engine primitives” that behave like **sequential breath** ($\$$) and **ratio steering** (φ). Nothing here requires an observer for the operations to run; the observer only supplies **readout** (digits, base, index).

D.1 Euler’s number e as a “breath” limit

Two core definitions:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!}$$
$$e = \lim_{m \rightarrow \infty} \left(1 + \frac{1}{m}\right)^m$$

The second form is the cleanest “engine” picture: a repeated compounding step indexed by an integer m .

A useful asymptotic (how fast it converges):

$$\text{Let } E_m := \left(1 + \frac{1}{m}\right)^m.$$

Using the Taylor expansion $\ln\left(1 + \frac{1}{m}\right) = \frac{1}{m} - \frac{1}{2m^2} + O\left(\frac{1}{m^3}\right)$,

we get $m \ln\left(1 + \frac{1}{m}\right) = 1 - \frac{1}{2m} + O\left(\frac{1}{m^2}\right)$,

so $E_m = e \cdot \exp\left(-\frac{1}{2m} + O\left(\frac{1}{m^2}\right)\right) = e\left(1 - \frac{1}{2m} + O\left(\frac{1}{m^2}\right)\right)$.

That is: the “gap” to e closes like $1/m$.

D.2 φ (golden ratio) as the “steer” recursion

Definitions / fixed points:

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

$$\varphi^2 = \varphi + 1$$

$$\varphi = 1 + \frac{1}{\varphi} \quad \Longleftrightarrow \quad \varphi = 1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{\ddots}}}$$

Fibonacci recursion as the operational generator:

$$F_0 = 0, \quad F_1 = 1, \quad F_{n+1} = F_n + F_{n-1} \quad (n \geq 1).$$

Then the steering ratio appears as the stable limit:

$$\frac{F_{n+1}}{F_n} \rightarrow \varphi \quad (n \rightarrow \infty).$$

Closed form (Binet), exposing φ as the growth eigenvalue:

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}, \quad \psi = \frac{1 - \sqrt{5}}{2} = -\varphi^{-1}.$$

So for large n :

$$F_n \approx \frac{\varphi^n}{\sqrt{5}} \quad \text{and therefore} \quad F_n^{-1} \approx \sqrt{5} \varphi^{-n}.$$

D.3 A concrete $e \leftarrow \varphi$ intertwine (Fibonacci-indexed breath)

If you want a **single recursive pipeline** where φ controls the “frame growth” and e is the “breath limit”, use:

1) Generate F_n via the Fibonacci recursion (steer). 2) Feed F_n into the compounding limit (breath):

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{F_n}\right)^{F_n}.$$

Call the Fibonacci-indexed approximants:

$$E_n^{(\varphi)} := \left(1 + \frac{1}{F_n}\right)^{F_n}.$$

Because $F_n \rightarrow \infty$, the limit is still e (this part is “observerless”: it is a property of the operation, not the label).

What φ changes is the convergence in n :

From the asymptotic in D.1, $e - E_n^{(\varphi)} \approx \frac{e}{2F_n}$.

Using $F_n \approx \varphi^n / \sqrt{5}$, $e - E_n^{(\varphi)} \approx \frac{e}{2} \cdot \frac{\sqrt{5}}{\varphi^n} = \left(\frac{e\sqrt{5}}{2}\right) \varphi^{-n}$.

So in the Fibonacci-indexed frame, the $e \leftarrow \varphi$ gap decays **exponentially in n** with decay rate φ^{-n} . That’s a clean “echo stack”: the **steer** (φ) sets the frame expansion, and the **breath** (e) emerges as the stabilized limit.

D.4 Spigot vs “random access” (sequencer vs teleport)

- **Spigot algorithms** output digits sequentially (a sequencer).
- **BBP-style digit extraction** can jump to digit n in certain bases (a teleport).

A simple, correct spigot for e (sequential decimal digits) follows from factorial-base carry propagation. It produces the digits after the decimal point in order:

```
def spigot_e(digits: int) -> str:
    # Sequential spigot for e: returns "2." + <digits> decimals
    n = digits + 5 # small safety buffer
    a = [1] * (n + 1)
    out = ["2", "."]

    for _ in range(digits):
        carry = 0
        for i in range(n, 0, -1):
```

```

x = a[i] * 10 + carry
a[i] = x % (i + 1)
carry = x // (i + 1)
out.append(str(carry))

```

```

return "".join(out)

```

By contrast, no comparably simple **BBP-type** (base- 2^k) “jump-to-digit” formula for e is currently standard/known in the way BBP is for π .

D.5 Minimal “engine” summary of the triad

- π : “carrier wave” via BBP-type digit extraction in base 16 (hex frame).
- e : “breath” via compounding limit and spigot-style sequential digits (time/step accumulation).
- φ : “steer” via fixed-point recursion and Fibonacci growth eigenvalue (frame scaling).

A tight e – φ bridge is:

$$F_{n+1} = F_n + F_{n-1} \quad \Rightarrow \quad \left(1 + \frac{1}{F_n}\right)^{F_n} \rightarrow e, \quad \text{with error} \sim C\varphi^{-n}.$$

End (v2).